Backend Server Creation

1. Create backend folder in our project:

course-management-system-main/

│

├── react-frontend/

├── mock-api/

└── backend/

2. Initialize Node.js Project
   In terminal: cd backend
   npm init -y
   This creates
   backend/
   └── package.json (we see the details of name, version and all)

3. Install Express:
   npm install express

4. **Install Development Tools**
   Install Nodemon: npm install --save-dev nodemon

   Install Environment Variable Support: npm install dotenv

   Install CORS: npm install cors

   Install Morgan: npm install morgan

5. **Create Backend Architecture:**

   backend/
   │
   ├── **config/**
   │
   ├── **controllers/**
   │
   ├── **middleware/**
   │
   ├── **models/**

```
│
├── routes/
│
├── services/
│
├── utils/
│
├── .env
├── .gitignore
├── package.json
└── server.js
```

i) Create Server.js

```
const express = require("express");

const cors = require("cors");

const dotenv = require("dotenv");

const morgan = require("morgan");


dotenv.config();


const app = express();



// -------------------------

// Middleware

// -------------------------


app.use(cors());


app.use(express.json());
```

```
app.use(express.urlencoded({
  extended: true
}));

app.use(morgan("dev"));



// -------------------------
// Test Route
// -------------------------

app.get("/", (req, res) => {

  res.json({
    message:
      "Student Course Management API is running"
  });

});



// -------------------------
// Server
// -------------------------

const PORT =
  process.env.PORT || 5000;
```

```
app.listen(PORT, () => {

  console.log(
    `Server running on http://localhost:${PORT}`
  );

});
```

6. **Add NPM Scripts**
   Open:
   backend/package.json
   Find:
   "scripts": {
     "test": "..."
   }
   Replace with:
   "scripts": {
     "start": "node server.js",
     "dev": "nodemon server.js"
   }
   Now we can run:
   npm run dev

---

7. **Start Backend**
   Inside backend: npm run dev
   You should see:
   [nodemon] starting `node server.js`

   Server running on
   http://localhost:5000

---

8. **Test Browser**
   Open:
   http://localhost:5000
   You should see:
   {
     "message":

```
    "Student Course Management API is running"
  }
```

If this works, your Express server is running.

9. . Create a Health Check API

A professional backend usually has a health check.

Add:

```
app.get("/api/health", (req, res) => {

    res.status(200).json({
        success: true,
        message: "Backend is healthy",
        timestamp: new Date().toISOString()
    });

});
```

Now:

```
http://localhost:5000/api/health
```

should return:

```
{
    "success": true,
    "message": "Backend is healthy",
    "timestamp": "..."
}
```

10. Create Our First Real Backend API

Create:

```
backend/data/
```

Then:

```
backend/data/courses.js
```

```
const courses = [
  {
    id: "1",
```

```
        courseName: "Python Programming",
        courseCode: "CS101",
        instructor: "Dr. Kumar",
        duration: "8 Weeks",
        category: "Programming",
        description:
            "Learn Python programming from beginner to advanced level.",
        status: "Active"
    },
    {
        id: "2",
        courseName: "Web Development",
        courseCode: "WD101",
        instructor: "Dr. Priya",
        duration: "10 Weeks",
        category: "Web Development",
        description:
            "Learn modern web development.",
        status: "Active"
    }
];

module.exports = courses;
```

Create Course Controller

Create:

backend/controllers/courseController.js

```
const courses =
    require("../data/courses");



// GET all courses


const getCourses = (req, res) => {


    res.status(200).json({
        success: true,
        count: courses.length,
        data: courses
    });


};



module.exports = {
    getCourses
};
```